NYC yellow taxi trip distances and OpenMP scaling

PHASE 1 REPORT / Ahmed Wael and Seif Amr / 3 November 2026

For January 2015, the combined metered distance of the retained yellow taxi trips was 1.38 times the combined straight-line distance. Computing that comparison by pickup hour took 425 ms sequentially and 77 ms with eight OpenMP threads once the cleaned records were in memory.

Dataset and coverage

The instructor-approved input is the January 2015 CSV from the Kaggle mirror of NYC Taxi and Limousine Commission yellow taxi records. Each of its 19-column rows describes a reported trip. The analysis uses pickup and drop-off times and coordinates, plus metered miles. Geographic and duration filters restrict the population represented below.

Processing stage	Records	Bytes
Raw CSV	12,748,986	1,985,964,692
Cleaned binary	12,435,056	596,882,688
100 MB prefix	2,083,333	99,999,984
500 MB prefix	10,416,666	499,999,968

How the data was cleaned

The preprocessing script reads 200,000 rows at a time. It parses both timestamps to second precision and converts the five numeric fields to numbers. Missing or unparseable values fail their field's test. The following counts assign each rejected row to its first failed test.

Test in execution order	Removed
January pickup and duration from 1 to 86,400 seconds	15,192
Both endpoints within 40–42° N and 75–72° W	250,559
Metered miles greater than 0 and at most 200	48,179

The pipeline retained 97.54% of the raw records. It preserves source order and repeated observations because these fields lack a unique trip ID. The two subsets are nested prefixes that end on complete records. They serve as timing inputs and may differ from a random sample of the month.

Page 6 lists the data sources and the repository with reproduction commands.

Hourly distance and the sequential baseline

What the computation measures

For each pickup hour from 00 through 23, the program accumulates the trip count, metered metres, direct metres and elapsed seconds. The direct distance is the great-circle distance between the recorded coordinates on a sphere with radius 6,371 km. It gives a common reference for trips of different lengths.

The baseline converts coordinate differences to radians and applies the haversine formula. For latitudes ϕ_1 and ϕ_2 and longitude difference $\Delta\lambda$, it computes $a = \sin^2(\Delta\phi/2) + \cos(\phi_1)\cos(\phi_2)\sin^2(\Delta\lambda/2)$. The central angle is twice the arcsine of $\sqrt{a}$. Multiplying by Earth's radius gives the distance in metres.

One international mile equals 1,609.344 metres. Each trip's metered and direct distances are rounded to the nearest metre before addition. The ratio divides the two sums, so longer trips have more influence. An individual trip can have coincident endpoints and still contribute to its hour.

What one pass produces

Retained trips	Metered km	Direct km	Ratio
All hours	56,083,265	40,551,101	1.383
19:00–19:59	3,208,762	2,346,611	1.367

The busiest pickup hour was 19:00–19:59, with 786,139 trips over the month. Across all retained trips, mean duration was 13.33 minutes. The distance gap includes street layout, route choice and measurement error. These records alone cannot apportion the gap among those causes.

Correctness checks

The sequential C++ loop reads one 48-byte record at a time from a contiguous vector. It updates one of 24 buckets using 64-bit integer sums. Rounding before reduction makes the result independent of the order in which threads combine their partial sums.

A separate NumPy calculation on the parsed CSV uses an atan2 form of the central angle and a weighted histogram by pickup hour. All 96 full-month output values matched the C++ baseline exactly. On each input size the benchmark also compared the parallel table to the sequential table and checked a checksum after every pass.

The reproducible hourly table is results/reference.csv. Kilometres above are rounded for display.

Work allocation and scheduling

How the threads share the work

Each trip can be measured independently. A single OpenMP parallel for loop assigns record indices to workers. A declared reduction gives each worker a private copy of the 24 hourly buckets, then adds the private counts and sums into the output array at the end of the loop.

Each private table occupies 768 bytes. Threads only read the input vector and accumulate locally during the scan. This avoids concurrent increments to the same popular hour, removes the need for a per-trip lock and limits synchronization to the end of the loop. Individual OpenMP tasks would add dispatch work to a regular loop.

Schedule comparison

Eight threads on the full month	Median	Range of five
Static contiguous blocks	77.2 ms	73.7–78.8 ms
Dynamic chunks of 1,024 trips	85.8 ms	80.7–130.2 ms
Guided chunks, minimum 1,024	87.7 ms	70.6–100.7 ms

Static scheduling had the lowest median. Every retained trip uses the same distance formula and a single bucket update, so there is little variation in work per record. Giving each worker one contiguous block preserves sequential access and avoids requesting more chunks. Guided produced the fastest individual pass, but its other passes brought the median above static.

Timing boundaries

Tests ran on Windows 11 Pro for Workstations with an Intel Core i5-11400H, six physical cores and twelve logical processors. GCC 16.2.0 from MSYS2 UCRT64 compiled a single C++17 executable with -O3 and -fopenmp. The serial path uses an ordinary loop. Dynamic team sizing was disabled, and the executable verified the requested team size.

For each configuration, the program loads and validates the binary file, warms up once, and times five more passes with steady_clock. Tables report the median elapsed time for the computation, including the parallel reduction. Preprocessing, loading, validation and printing are excluded. Configurations were run in sequence with default OS thread placement.

Fixed workload performance

The serial median for each input provides its own baseline. The table reports static OpenMP results. Speedup $S = T_s / T_p$ and efficiency $E = S / p$, where p is the number of OpenMP threads. The effective serial fraction f is defined on page 5.

Input	p	T_s (ms)	T_p (ms)	S	E	f
100 MB	1	72.2	73.1	0.99	98.7%	—
100 MB	2	72.2	42.4	1.70	85.0%	17.59%
100 MB	4	72.2	19.5	3.69	92.3%	2.76%
100 MB	8	72.2	13.5	5.33	66.6%	7.17%
500 MB	1	324.0	319.6	1.01	101.4%	—
500 MB	2	324.0	168.6	1.92	96.1%	4.09%
500 MB	4	324.0	98.7	3.28	82.1%	7.28%
500 MB	8	324.0	64.9	4.99	62.4%	8.60%
Full	1	425.0	381.2	1.12	111.5%	—
Full	2	425.0	208.2	2.04	102.1%	-2.03%
Full	4	425.0	113.3	3.75	93.8%	2.22%
Full	8	425.0	77.2	5.51	68.8%	6.47%

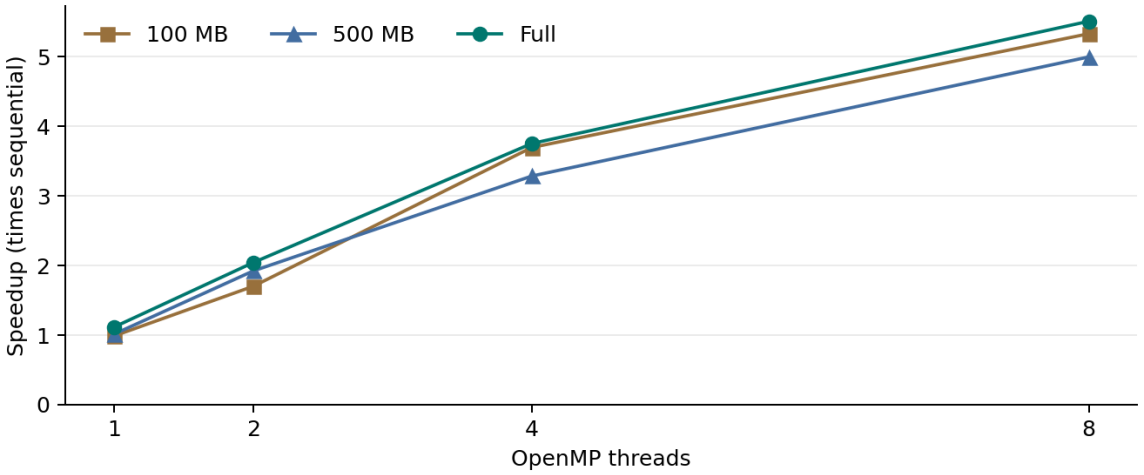


Figure 1. Each curve uses its own fixed input. All three inputs ran fastest with eight threads among the tested counts. The full input contains 12,435,056 trips.

Scaling limits and larger workloads

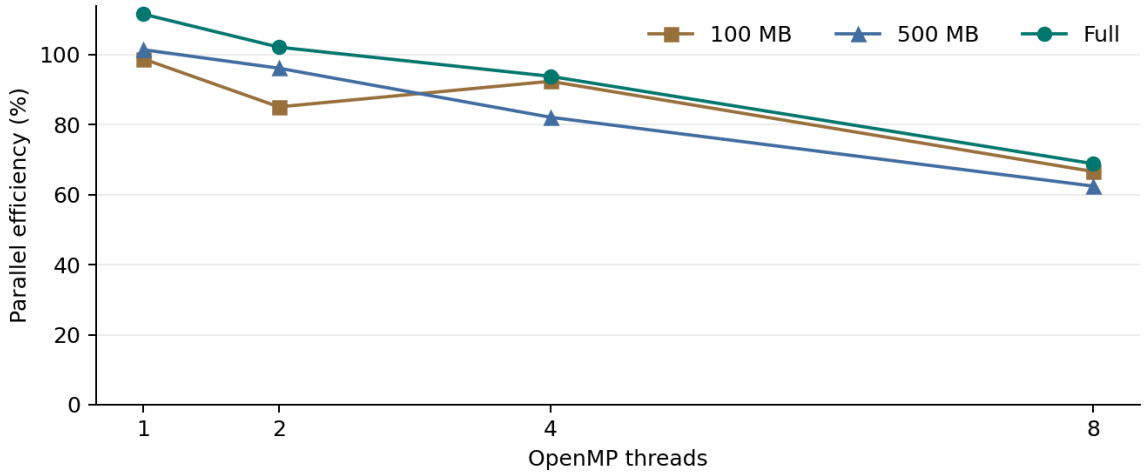


Figure 2. Efficiency relative to the separately measured C++ sequential path.

Amdahl's Law on a fixed month

The effective serial fraction f = (1/S − 1/p) / (1 − 1/p) was −2.03%, 2.22% and 6.47% at two, four and eight threads on the full input. It is undefined at one thread. A separate serial baseline, clock variation and differences between loop implementations can yield efficiencies over 100%. The negative estimate has no interpretation as a physical serial fraction.

At eight threads, the model S(p) = 1 / [f + (1 − f)/p] implies a ceiling near 15.5× if f stayed fixed. The changing estimates show why that ceiling is only illustrative. Eight workers already exceed the six physical cores and must share some execution resources.

Gustafson's Law with growing input

A second experiment assigned 1,554,382 trips per worker by growing the input prefix. In Gustafson's model, scaled speedup is p − α(p − 1), where α is the serial share of elapsed time on the scaled parallel run. If that share is small, a larger job can approach p-fold throughput.

Threads	Trips processed	Median	Throughput gain[1]
1	1,554,382	50.2 ms	1.00×
2	3,108,764	55.7 ms	1.80×
4	6,217,528	65.5 ms	3.07×
8	12,435,056	75.8 ms	5.29×

[1] Trips per second relative to one OpenMP worker. The eight-worker run processed eight times as many trips in 1.51 times the elapsed time. This throughput ratio is an empirical comparison across input sizes. The experiment did not independently time α.

Locality and remaining bottlenecks

The cost of changing the access order

A single-thread stride test sums the elapsed-seconds field of every trip. For stride k, it visits indices 0, k, 2k and so on, then repeats for offsets 1 through k − 1. All passes returned the same 9,944,008,441 seconds. Only the traversal order changed.

Stride in records	Address step	Median	Relative time
1	48 bytes	37.4 ms	1.00×
8	384 bytes	77.7 ms	2.08×
64	3,072 bytes	84.1 ms	2.25×

The contiguous scan can reuse adjacent cache lines and is easier to prefetch. At stride 64, the scan jumps 3 KB between reads and revisits nearby records much later. The slowdown is consistent with weaker spatial locality. Cache misses were inferred from the access pattern and timings rather than counted by hardware.

Profile and end to end costs

A separate GNU gprof build used -pg and disabled inlining. Across nine full serial passes, measure() was called 111,915,504 times and accounted for 27.24% of sampled self time. The profiling hooks accounted for 55.56%, so the instrumented percentages identify hot code but cannot be read as release-build timings. The reduction's private tables avoid per-trip shared counter writes. The tests do not isolate the cost of the final merge or establish a false-sharing miss rate.

At eight threads the full file took 300 ms to allocate, read and validate, versus 77 ms to compute. Using the observed load times gives about 1.88× from load through computation, compared with 5.51× for the kernel alone. Reusing a loaded dataset would amortize that cost across several queries. A packed duration column is a candidate for the stride-only query, which reads four useful bytes from each 48-byte record.

Sources and reproducibility

Dataset mirror: kaggle.com/datasets/elemento/nyc-yellow-taxi-trip-data
Publisher: nyc.gov/site/tlc/about/tlc-trip-record-data.page
Code and run instructions: github.com/shadow9-1-1/Taxi-Analytics

README.md gives the preprocessing, build, benchmark and profiling commands. The results directory contains raw trials, medians, the scaled-workload experiment and the gprof output.